

DUNE: An Indoor Ultra-Wideband Positioning System

A record of the work, the reasoning, and the dead ends

DUNE project — UWB_3

22 July 2026

Abstract

This report describes a system that measures where a small rover is inside a room, to within a few centimetres, using radio distance measurements. Five fixed radio “anchors” sit around the room. One or two “tags” ride on the rover. Each tag measures its distance to each anchor several times a second, and a computer turns those distances into a position. A ceiling camera watching a printed marker on the rover provides an independent, trusted measurement that we use to check our answers.

The report is written to be read, not decoded. Equations appear only where they make something clearer, and each one is explained in plain words. The final section is a deliberately honest list of everything we tried that did not work, because those attempts shaped the design as much as the successes did.

Contents

1	What the system does, in one page	2
2	Ground truth: making the ceiling camera trustworthy	2
2.1	How high is the camera, really?	2
2.2	Lens distortion: deliberately not corrected	3
2.3	Tying the camera to the room	3
3	Making the radios measure distance correctly	3
3.1	How a distance measurement works	3
3.2	Antenna delay: the single biggest error	3
3.3	An unexplained anomaly	4
3.4	Strong signals read long	4
4	From distances to a position	4
5	Static accuracy: how good is it standing still?	5
6	Using time: the Kalman filter	5
7	The stubborn residual: a slow “breathing”	6
7.1	Is it caused by irregular measurement timing?	6
7.2	Is it the tag, or the links?	6
7.3	Looking at the radio signal directly	6
7.4	What we did about it	7

8	A different estimator: moving-horizon estimation	7
8.1	Using the IMU without trusting it	8
9	Two tags: estimating the whole rover	8
9.1	Checking it before trusting it	8
10	Getting the data off the rover	8
11	Driving the rover: truth and radio together	9
11.1	First result	9
12	What was tried and did not work	10
12.1	Using the IMU's acceleration to predict motion	10
12.2	Feeding individual distances to the filter instead of positions	10
12.3	Machine learning for the signal-strength correction	10
12.4	Absorbing the drift into the filter's internal states	10
12.5	A safety guard that made things worse	10
12.6	Two independent tag estimates	11
12.7	Tuning the motion model for smoothness	11
12.8	The gyroscope turn model showed no benefit — yet	11
12.9	Camera lens distortion correction	11
12.10	Old calibration values surviving reprogramming	11
12.11	A self-inflicted performance collapse	11
12.12	Deferred rather than failed	11
13	Where things stand	12

1 What the system does, in one page

A radio tag on the rover talks to five fixed radio anchors. By carefully timing how long a radio message takes to fly to an anchor and back, the tag works out how far away that anchor is. Radio travels about 30 centimetres every nanosecond, so timing must be accurate to a fraction of a nanosecond for centimetre results. This is what ultra-wideband (UWB) radio hardware is designed to do.

Once the tag knows its distance to several anchors, finding its position is a geometry problem. Each distance says “I am somewhere on a circle of this radius around that anchor.” With three or more circles, the point where they all nearly meet is the answer. They never meet perfectly, because every distance has some error, so we look for the point that disagrees with all the measurements as little as possible.

That single-shot answer is noisy. So we add a second layer that uses time: the rover cannot teleport, so a position that jumps a metre between two readings taken a fifth of a second apart is wrong. Filters that encode this idea (a Kalman filter, and later a moving-horizon estimator) smooth the answer and reject nonsense.

Finally, because a rover has two tags mounted a fixed distance apart on a rigid plate, we can do better still. The two tags cannot move independently. Using that fact, we estimate the pose of the whole rover — position *and* which way it is facing — rather than two separate points.

Table 1: The pieces of the system.

Piece	Count	Role
Anchors (DW1000 + ESP32)	5	Fixed around the room, at 24 cm height
Tags	2	On the rover, 52.7 cm apart; one carries an IMU
Ceiling camera (Kinect v2)	1	Watches a printed marker: independent truth
Host computer (MATLAB)	1	Does all position calculation and logging

2 Ground truth: making the ceiling camera trustworthy

Before trusting any radio result, we needed something better to compare against. A camera on the ceiling looking down at a printed marker can do this, but only after it is calibrated. A camera that does not know its own height or lens properties will report confident, wrong positions.

2.1 How high is the camera, really?

The camera’s height matters because it sets the scale: the same marker at twice the height looks half as big. We measured it by *parallax*. Place an object on the floor, note where it appears in the image; raise it by a known amount, note where it appears now. Because the camera is not exactly overhead, raising an object makes it appear to shift sideways, and how much it shifts depends on the height of the camera.

This took three attempts, and the first two were wrong:

- A first estimate of 3.873 m came from a single noisy session and was unreliable.
- Two careful repeat sessions, hopping a box of known thickness around all four quadrants, both gave about 4.21 m. Reproducible, but still biased, because lens distortion inflates this kind of measurement by a few percent.
- The final method removed that bias by working in real-world coordinates rather than raw pixels. It gave **4.066 m**, with a fit residual of 7.7 mm.

The same procedure produced the camera’s focal length, 1067.6 pixels. As a sanity check, the ratio of height to focal length must match an independent measurement of how many metres one pixel covers on the floor. It agreed to within 0.2%, which is what convinced us the numbers were finally right.

2.2 Lens distortion: deliberately not corrected

We photographed straight strings and measured how much they bowed in the image. The bow was under 1.5 pixels across the whole field — small enough to ignore. When we tried to fit a distortion model anyway, it produced a nonsensical lens centre because it was fitting our clicking noise rather than any real distortion. We recorded the lens as distortion-free and moved on. This is worth stating plainly: correcting a model that does not need correcting made things worse, not better.

2.3 Tying the camera to the room

The camera also needs to know where the room is. We placed a marker at several known positions and solved for the transformation between camera coordinates and room coordinates. The fit had an error of **13.5 mm** (worst point 18.7 mm).

One detail cost real time: the printed marker is 17.78 cm across, but the detector consistently behaved as though it were **16.9 cm**. This is a property of how the detector defines the marker’s edges. Once anchored against tape measurements on the floor, the corrected size made the camera and the tape agree.

The payoff is that the camera became a measuring instrument: click any point in the image whose height is known, and get its room coordinates. We used this to place the anchors themselves — clicking each anchor in the image rather than measuring five positions by hand.

3 Making the radios measure distance correctly

3.1 How a distance measurement works

The tag sends a message; the anchor replies; the tag times the round trip. The catch is that the anchor takes some time to reply, and that reply delay is not perfectly known. The scheme used here (a double-sided exchange) cancels most of that unknown by using timestamps from both sides:

$$t_{\text{flight}} = \frac{T_{\text{round}}^{\text{tag}} \cdot T_{\text{round}}^{\text{anc}} - T_{\text{reply}}^{\text{tag}} \cdot T_{\text{reply}}^{\text{anc}}}{T_{\text{round}}^{\text{tag}} + T_{\text{round}}^{\text{anc}} + T_{\text{reply}}^{\text{tag}} + T_{\text{reply}}^{\text{anc}}}, \quad d = c \cdot t_{\text{flight}} \quad (1)$$

In words: each side measures both how long it waited and how long it took to reply, and the combination cancels the error that arises because the two devices’ clocks run at slightly different speeds. We audited this formula specifically to rule out clock drift as a source of our residual error, and it is clean.

3.2 Antenna delay: the single biggest error

The radio signal takes time to travel through the antenna and circuitry before it leaves the device. To the timer this looks exactly like extra distance. Each board needs a calibration constant for this delay.

We hit a trap here worth recording. The calibration constants are stored in the anchors’ flash memory, and **that memory survives reprogramming**. Old, wrong values from an earlier era

of the project were still present. One anchor reported distances 4 m too long, another 1 m too long, consistently and quietly. The symptom was that the computed position sat outside the room. A factory reset of the stored values fixed it instantly.

We then calibrated properly: park the tag at a spot the camera has measured, compare each anchor’s reported distance to the true distance, and adjust that anchor’s delay constant. One tick of the delay setting equals 4.69 mm, so the correction is direct. Two rounds brought every anchor within 16 mm, and the parked position landed **4 mm** from the camera’s answer.

3.3 An unexplained anomaly

While calibrating we found something we never fully explained. The same distance measurement code gives different answers depending on how rapidly it is called. Rapid back-to-back measurements and the normal round-robin pattern disagree by a fixed amount per anchor, between 60 mm and 500 mm. Both use the identical routine.

We could not identify the cause. Our workaround was to calibrate using the same pattern the system actually uses in normal operation, so the anomaly cancels. It is recorded here as an open question, not a solved one.

3.4 Strong signals read long

After delay calibration, a clear pattern remained: when a signal arrives strong, the measured distance is slightly too long; when weak, slightly too short. This is a known characteristic of this radio chip. Across a 19-position measurement campaign, the correlation between signal strength and distance error was +0.78.

We fitted a straight-line correction per anchor,

$$d_{\text{corrected}} = d_{\text{measured}} - (a_k \cdot p_{\text{fp}} + b_k), \quad (2)$$

where p_{fp} is the strength of the first arriving path of the signal and a_k, b_k are that anchor’s fitted constants. Slopes came out between 7.7 and 9.5 mm per decibel. This correction cut the position error roughly in half (199 mm to 94 mm, see Section 5).

We also tried fitting this relationship with a neural network and with a physics-constrained network. Both did *worse* than the straight line. That result is discussed in Section 12.

4 From distances to a position

Given corrected distances d_k to anchors at known positions $\mathbf{a}_k$, we look for the position $\mathbf{p}$ that best explains them. We minimise the total squared disagreement:

$$\hat{\mathbf{p}} = \arg \min_{\mathbf{p}} \sum_k w_k \left(\underbrace{\|\mathbf{p} - \mathbf{a}_k\|}_{\text{distance the guess implies}} - \underbrace{d_k}_{\text{distance measured}} \right)^2 \quad (3)$$

Each term asks: if the rover were at $\mathbf{p}$, how far off would this anchor’s measurement be? We add these up and pick the $\mathbf{p}$ that makes the total smallest. The weights w_k let us trust some anchors more than others.

Because the distance function is curved rather than straight, there is no formula for the answer; we start from a guess and improve it repeatedly (Levenberg–Marquardt). The height of the tag is fixed and known, so only two numbers are being solved for, which makes it fast and reliable.

Two practical additions matter:

- **Rejecting outliers.** After a first solve, any measurement that disagrees wildly with the rest is dropped and the solve is repeated. The threshold adapts to how noisy that particular reading set is.
- **Refusing bad geometry.** If the only anchors answering happen to lie almost in a straight line, the geometry is ambiguous — there are two mirror-image answers and no way to choose. The solver detects this and declines to answer rather than produce a confident guess. This turned out to matter (see Section 12).

5 Static accuracy: how good is it standing still?

We parked the tag at 19 measured positions around the floor and recorded about 7,600 readings. This is the honest baseline.

Table 2: Position error over 19 parked positions, single readings, no time filtering.

	RMSE	Median	95th percentile
Raw (delay calibration only)	199 mm	132 mm	393 mm
With signal-strength correction	94 mm	69 mm	160 mm
Best possible with a perfect fixed correction	72 mm	31 mm	—

The last row deserves attention. We asked: suppose we cheated, and gave every position a perfect constant correction derived from its own data. How good could it possibly get? The answer was 72 mm — not zero. That tells us the remaining error is not a fixed map of the room that we simply have not measured well enough. It *changes over time at a fixed position*. No static correction can remove it. This single result redirected the whole investigation described in Section 7.

6 Using time: the Kalman filter

A single reading is noisy, but the rover moves smoothly. A Kalman filter combines a prediction of where the rover should be with the new measurement, weighted by how much each is trusted.

The filter tracks position and velocity, $\mathbf{x} = [p_x, p_y, v_x, v_y]$, and between readings assumes velocity roughly persists:

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_k \Delta t, \quad \mathbf{v}_{k+1} = \mathbf{v}_k + (\text{unknown acceleration}). \quad (4)$$

How much unknown acceleration we allow is the main tuning knob: allow a lot and the filter chases noise; allow little and it lags behind real motion.

Several additions were needed, each fixing an observed problem:

Outlier handling that bends rather than breaks. Originally a reading that looked too surprising was rejected outright. This is brittle. Instead, moderately surprising readings are now *down-weighted* in proportion to how surprising they are, and only wildly impossible ones are rejected. This alone improved one operating mode from 164 mm to 105 mm.

Knowing when it is stopped. When the IMU says the rover is still, we feed the filter an extra fact: the velocity is zero. Useful, but not sufficient on its own — see the next point.

Freezing the noise budget when still. Even with zero velocity enforced, the filter kept re-injecting position uncertainty every step, which kept it responsive to noise and made a parked tag wander. Reducing the assumed acceleration noise while stationary is what actually made the parked dot settle.

Remembering each anchor’s personality. Each anchor has a small persistent bias. When the set of answering anchors changes — one drops out for a moment — the average of those biases changes, and the position jumps. We added one slowly varying bias value per anchor, so the model becomes $d_k = \|\mathbf{p} - \mathbf{a}_k\| + b_k$. These are remembered through dropouts. Position jumps at anchor changes fell from 13–30 mm to 6–9 mm.

Holding a missing anchor’s last value. Complementary idea: if an anchor misses a reading, reuse its recent distance briefly rather than dropping it, so the geometry stays complete. Jumps fell from 13–28 mm to 6–10 mm.

7 The stubborn residual: a slow “breathing”

With everything above working, a parked tag still drifted slowly by 5–7 cm over seconds to minutes. Fast jitter was gone; this was different. It looks exactly like slow genuine motion, which means no filter can remove it — a filter that removed it would also refuse to follow the rover when it actually crept forward.

We investigated systematically. The value of this section is as much in what we *eliminated* as in what we found.

7.1 Is it caused by irregular measurement timing?

Hypothesis: the measurement rate wobbles (retries, skipped anchors), and the radio gives slightly different answers depending on timing — which the anomaly in Section 3.3 suggests. We tested this on 44,000 stored measurements by correlating each anchor’s realised timing gap against its distance error.

Rejected. Correlation never exceeded 0.06. A real but tiny effect exists (± 5 –7 mm depending on whether the previous slot succeeded), but it is far too small. One useful by-product: the first reading after an anchor returns from a long absence reads about 39 mm long, which justified keeping the “hold last value” logic.

7.2 Is it the tag, or the links?

If the cause were in the tag (its clock, its temperature), every anchor’s distance would drift *together*. If it is in the individual radio paths, they drift independently. This is a clean, decisive test.

We measured how each anchor’s error correlated with the others across 23 parked periods. Average correlation: **+0.05** — essentially independent.

Conclusion: the cause is per-link, not tag-side. That eliminated the tag’s clock, temperature, and battery in one step, and pointed at the radio path itself.

7.3 Looking at the radio signal directly

We added the ability to capture the raw shape of the arriving radio signal. The result was unambiguous. The first arriving path — the direct line, the one whose timing we use — is roughly **20 dB weaker than the strongest reflection**, and only 8–10 dB above the noise floor.

This is the expected situation when antennas sit 24 cm above the floor: the signal grazes along the ground, the floor reflection arrives almost simultaneously with the direct path, and the two blend together. The exact blend shifts as the environment changes, moving the apparent arrival time by centimetres. A quiet, empty room drifted just as much as one with someone walking through it, which shows this is a property of the geometry, not of moving people.

7.4 What we did about it

Understanding the cause did not immediately remove it: the fix is physical (raise the antennas), and the anchors are permanently installed. So we did two things.

First, we accepted it and documented it. Second, since a parked rover is genuinely stationary, we suppress the *displayed and logged* drift rather than pretend to fix the measurement. When the system is confident the rover is stopped, the reported position is frozen inside a 5 cm deadband; sustained real movement releases it. While moving, a light smoothing filter removes the high-frequency jitter.

We are explicit that this is cosmetic. The raw estimate is still recorded for analysis. It makes a parked rover look correct, which it is, without claiming the underlying distances improved.

8 A different estimator: moving-horizon estimation

A Kalman filter looks at one reading at a time and cannot revisit past decisions. A moving-horizon estimator (MHE) instead keeps the last N readings and re-solves the whole recent trajectory each time. It can reinterpret the recent past in light of new evidence.

We minimise, over the entire window at once:

$$\min_{\{\mathbf{x}_k\}} \underbrace{\sum_k \sum_j w_{kj} \rho(\|\mathbf{p}_k - \mathbf{a}_j\| - d_{kj})}_{\text{fit the measurements}} + \underbrace{\sum_k \|\mathbf{x}_{k+1} - f(\mathbf{x}_k)\|_Q^2}_{\text{obey the motion model}} + \underbrace{\|\mathbf{x}_1 - \bar{\mathbf{x}}\|_P^2}_{\text{remember what came before}} \quad (5)$$

The three parts read as: match what the radios measured; do not move in ways the rover physically cannot; and stay consistent with what we already believed before this window began. The function ρ is a robust loss — it grows more gently than a square for large errors, so a single bad measurement bends the answer slightly instead of dragging it.

Implemented with CasADi and IPOPT, it solves in about 3 ms, comfortably fast enough.

Table 3: Moving-horizon estimator versus Kalman filter, same recorded data.

	Kalman	Moving horizon
Path smoothness while moving (lower better)	0.61	0.36
Lag behind raw measurements	22 mm	28 mm
Parked wander (median over positions)	54 mm	55 mm
Position error, parked	64 mm	61 mm

The moving-horizon estimator is clearly better while moving — about 40% smoother for 6 mm more lag — and a tie when parked. It is offered as an option rather than made the default, because it has one wart: at two of nineteen parked positions it drifts about 0.23 m, as the slow breathing pulls the whole window along. Its average position stays correct, so this is drift, not bias.

A safety mechanism proved necessary. The optimiser occasionally settles into a different-but-similar solution between steps, causing the answer to jump. We detect this — an output that leaps further than physics allows — and fall back to the measurement-only answer. The first version of this guard fell back to *dead reckoning* instead, which is discussed in Section 12 because it failed spectacularly.

8.1 Using the IMU without trusting it

The tag carries an IMU. We measured what it could actually be trusted for:

- Its acceleration bias is essentially zero — good.
- Its sense of which way is “north” is rotated by an unknown amount — not trustworthy.
- The tag lies flat, so its measured turn rate is directly the rover’s turn rate.

This suggested using *changes* rather than absolute values. A gyroscope measuring “turning left at 30 degrees per second” needs no compass and no reference direction. So the motion model was extended to let the velocity direction rotate at the measured turn rate. When the turn rate is zero this reduces exactly to the previous model, so it is safe by construction.

9 Two tags: estimating the whole rover

The rover carries two tags on a rigid plate, 52.7 cm apart. They cannot move independently, and that is valuable information.

Rather than computing two positions and comparing them, we estimate one rover pose: its centre, its heading ψ , and its speed. Both tags are then *derived* from that pose:

$$\mathbf{p}_{\text{front}} = \mathbf{c} + \frac{L}{2} \begin{bmatrix} \cos \psi \\ \sin \psi \end{bmatrix}, \quad \mathbf{p}_{\text{rear}} = \mathbf{c} - \frac{L}{2} \begin{bmatrix} \cos \psi \\ \sin \psi \end{bmatrix}. \quad (6)$$

This is a deliberate design choice with three consequences, all free:

1. The distance between the tags is **exactly** 52.7 cm in every answer, because the equations cannot express anything else. It is not a penalty that can be traded away.
2. “Both tags are stopped, or both are moving” is automatic, since there is only one speed.
3. Heading comes out of the geometry — which tag is where — *combined* with the gyroscope’s turn rate. Neither source alone is as good.

Because the rover is a car with front steering, it cannot slide sideways. We encoded that too, by defining velocity as speed along the heading. Again this is structural: sideways motion is not representable.

The two tags report at different instants. Rather than pairing them up in time, each reading becomes its own entry in the window, tagged with which end of the rover it came from. No interpolation is needed.

Ground slope is handled using the IMU’s roll and pitch directly. On a slope the two tags are no longer at the same height, and their horizontal separation shrinks to $L \cos(\text{pitch})$. Both effects are included.

9.1 Checking it before trusting it

Since a sign error here would be invisible but corrupting, we tested against simulated data with a known answer before using real measurements: a simulated rover driving an arc over the real anchor layout, with realistic 30 mm distance noise.

10 Getting the data off the rover

Originally each tag was connected by USB cable. With two tags on a moving rover this does not work.

Table 4: Rigid two-tag estimator on simulated data with known truth.

Condition	Centre error (median)	Heading error	Tag separation
Flat ground	14.1 mm	1.18°	0.5000 m (exact)
8.6° slope	14.1 mm	1.17°	matches $L \cos \theta$
−11.5° slope	14.2 mm	1.18°	matches $L \cos \theta$

The tags were already broadcasting every reading over WiFi; only the receiving side was missing. A long-standing project note claimed the network blocked this kind of traffic. We tested it directly rather than trusting the note, and **the note was wrong** — both tags reached the computer over WiFi with no changes to the radios at all.

One practical detail: the obvious MATLAB tool for receiving this traffic requires a toolbox we do not have. We used the Java networking classes that ship with MATLAB instead. This cost us later in an unexpected way (Section 12).

11 Driving the rover: truth and radio together

The final piece connects everything. The rover already carries a printed marker that the ceiling camera tracks. So driving the rover with both tags aboard captures, on one clock: the true position from the camera, and the radio measurements from both tags.

We adapted an existing rover teleoperation program (driven by a PS4 controller) to also record the radio data. One design decision is worth stating: the position estimator does *not* run inside the driving loop. That loop is responsible for the emergency stop, and the optimiser occasionally takes 90 ms. Delaying an emergency stop to compute a position is a bad trade. The loop only records; the estimator runs afterwards on the recording, which gives an identical result.

11.1 First result

Table 5: First driven run, 212 seconds. Radio estimate versus camera truth.

Frame alignment found	−3.6° rotation, 0.46/0.59 m offset
Median error	98 mm
RMSE	268 mm

The trajectory shape matches the camera truth well, which confirms the whole chain works. But the numbers are **not yet trustworthy**, and it would be wrong to quote them as system accuracy. Three data quality problems dominate:

1. The camera only saw the marker 58% of the time, with a long gap. The largest apparent errors fall in stretches where there was no truth to compare against.
2. One of the two tags reported far less often than the other (386 readings versus 1124). The rigid estimator determines heading by seeing *both* ends, so heading was weakly constrained. The safety guard fired on 46% of solves.
3. On average only 3.6 of 5 anchors answered each reading, and the worst errors occurred near the edge of the anchor layout where the geometry is poorest.

These are fixable setup problems, not algorithmic ones. Fixing them is the current work.

12 What was tried and did not work

This section is deliberate. Several of these consumed more time than the successes, and a reader repeating this work should know where the holes are.

12.1 Using the IMU’s acceleration to predict motion

Feeding accelerometer data into the filter to predict where the rover was heading caused divergence of 6–14 metres. The cause was misdiagnosed for a long time as accelerometer bias. It was not: measurement later showed the bias is essentially zero. The real problem is that the IMU’s idea of which direction is which is rotated by an unknown angle, so accelerations were applied in the wrong direction. Left disabled. Using only the gyroscope’s turn rate, which needs no reference direction, works.

12.2 Feeding individual distances to the filter instead of positions

In principle, updating the filter with each distance separately is better than collapsing them into a position first — it works with fewer than three anchors. In practice it failed badly, giving 2244 mm error against 335 mm. Because the filter accepts each distance separately, it can satisfy one or two of them while sitting at a completely wrong position, and never notices. It is left available but not default.

12.3 Machine learning for the signal-strength correction

We fitted the relationship between signal strength and distance error with a neural network, and with a physics-informed network constrained to be monotonic. Tested honestly, by holding out entire positions, the plain straight line won:

Model	Position error
Straight line	98 mm
Neural network	107 mm
Physics-informed network	108 mm

The relationship really is a straight line over the range that occurs; the extra capacity only fitted noise. With 31,500 samples but only 19 distinct positions, the effective sample size was 19, not 31,500.

12.4 Absorbing the drift into the filter’s internal states

Since a parked rover is stationary, we tried inverting the filter’s assumptions while stopped: trust the position completely, and let the per-anchor bias values absorb the drift. This made things dramatically worse, 91 mm to 319 mm. The reason is instructive: the biases learned at one position are wrong at the next, and because they are deliberately remembered, the error propagates. This confirmed that the drift cannot be absorbed inside the estimator and must be handled at the output, or fixed physically.

12.5 A safety guard that made things worse

The guard protecting the moving-horizon estimator from occasional jumps originally fell back to dead reckoning — continuing at the last known velocity. This created a runaway: once it diverged, the real answer looked like a jump, so the guard engaged again, and the estimate walked 9 metres away. It fired on 73% of solves. The fix was to fall back to the measurement-only position, which cannot run away. The trip rate fell to 3%. General lesson: a fallback must be anchored to measurements, never to prediction alone.

12.6 Two independent tag estimates

Our first two-tag attempt tracked each tag separately. During one run both tags' displays slid several metres apart while physically 30 cm apart. Diagnosis found two causes: two anchors had lost power, leaving three nearly in a line, so the solver correctly refused to answer; and with no answer, the display followed the filter's dead-reckoned guess instead of stopping. This work was rolled back and replaced by the rigid single-pose formulation, which cannot express the two tags drifting apart. The display now freezes and says so when measurements stop.

12.7 Tuning the motion model for smoothness

To smooth a jagged track, the obvious knob is the filter's assumed acceleration noise. Sweeping it from 0.8 to 0.1 gave only 13% improvement while tripling the lag. The reason is that most of the roughness is the slow breathing, which sits in the same frequency range as real motion; no filter can separate them. A light smoothing filter on the *output* achieved 50% improvement for 25 mm of lag.

12.8 The gyroscope turn model showed no benefit — yet

Adding gyroscope-driven turning produced no measurable improvement (0.357 versus 0.362). This is not evidence it is useless: the test data was a hand-carried tag doing stop-and-go straight moves with no sustained turns. It remains unvalidated rather than disproven, pending a rover driving real arcs.

12.9 Camera lens distortion correction

Fitting a distortion model to a lens with less than 1.5 pixels of distortion produced a nonsensical lens centre, because it was fitting clicking noise. Skipped deliberately.

12.10 Old calibration values surviving reprogramming

Reprogramming the anchors does not erase their stored calibration constants. Wrong values from months earlier were silently in use, causing metre-scale errors that looked like a software bug. Any future rebuild should explicitly clear this memory first.

12.11 A self-inflicted performance collapse

When the radio recording was added to the rover driving program, the controller became unresponsive. The cause was mine, not the hardware. Three mistakes compounded: the large recording arrays were placed inside a structure that MATLAB copies on every loop iteration; the network socket was polled twice per iteration, each poll costing 15 ms because it waits for a timeout exception; and a convenient timestamp function cost 0.64 ms per call. Together these slowed the loop from 20 Hz to 1.4 Hz. Button presses last about 200 ms, so at 1.4 Hz most were simply never observed. The loop now reports its own rate and warns when it is too slow to catch input.

12.12 Deferred rather than failed

Two items were designed but set aside: a broadcast-based ranging protocol that would measure all anchors simultaneously (it requires reprogramming every anchor), and raising the antennas to escape the ground-reflection problem (the anchors are permanently mounted). Both remain the most promising routes to reducing the residual drift at its source.

13 Where things stand

The system is built and works end to end. Distances are calibrated, positions are computed and filtered, two tags give a full rover pose including heading, data reaches the computer wirelessly, and a driven rover records radio measurements and camera truth together for scoring.

The remaining error is understood rather than mysterious. It comes from antennas mounted low, where the direct signal path and its floor reflection blend together. This is a physical arrangement problem, and we know what would fix it.

The honest current numbers are: about 94 mm for a single reading standing still, 60–65 mm after filtering, and a driven-run figure that is not yet meaningful because the reference data was incomplete. Improving that measurement — not the algorithms — is the immediate next task.